

CLDP (Custom Lightweight Discovery Protocol) Specification

Dadi Sasank Kumar (22CS10020)
Assignment-7

March 30, 2025

Contents

1	Introduction	2
2	Protocol Overview	2
2.1	Message Types	2
2.2	Custom Header Format	2
3	IP Packet Crafting and Parsing	3
3.1	Raw Socket Programming	3
3.2	Packet Assembly and Parsing	3
4	Server Implementation	3
5	Client Implementation	3
6	Demonstration Scenario	4
7	Build and Run Instructions	4
7.1	Compiling the Code	4
7.2	Running the Server and Client	4
7.3	Capturing Traffic	4
8	Assumptions and Limitations	5
9	Conclusion	5

1 Introduction

This document specifies the Custom Lightweight Discovery Protocol (CLDP) implemented using raw sockets in C (POSIX standard). The protocol is intended for a closed network environment where nodes announce their presence and allow querying for application-level metadata such as CPU load, system time, or hostname.

2 Protocol Overview

CLDP operates directly on the IP layer using raw sockets with a custom protocol number (253). The protocol does not use any transport layer protocols like TCP or UDP, and the IP header is manually constructed including checksum calculation.

2.1 Message Types

The protocol supports the following message types:

- **HELLO (0x01):** Used for server nodes to announce their presence every 10 seconds.
- **QUERY (0x02):** Used by the client to request metadata. The implementation supports at least three types of metadata:
 - Hostname
 - System Time (timestamp)
 - CPU Load (or an alternative performance counter)
- **RESPONSE (0x03):** Used by servers to reply to QUERY messages with the requested metadata.

2.2 Custom Header Format

Each CLDP packet contains a custom header with a minimum length of 8 bytes. The header fields are as follows:

- **Message Type (1 byte):** The type of message (e.g., 0x01 for HELLO).
- **Payload Length (1 byte):** Length in bytes of the payload following the header.
- **Transaction ID (2 bytes):** Identifier for matching queries with responses.
- **Reserved (4 bytes):** Reserved for future use; currently set to 0.

Additional data (such as hostname strings or timestamps) are included immediately after the header.

3 IP Packet Crafting and Parsing

3.1 Raw Socket Programming

CLDP is implemented using raw sockets (`AF_INET`, `SOCK_RAW`) in Linux. Since raw socket programming requires manual IP header creation:

- The protocol number is set to 253 in the IP header.
- `IP_HDRINCL` is enabled using `setsockopt()`.
- A checksum is calculated using a standard IP checksum algorithm.

3.2 Packet Assembly and Parsing

- **Assembly:** The sender constructs the IP header and appends the CLDP header and the payload. For example, a HELLO message includes a node identifier string.
- **Parsing:** Incoming packets are filtered by checking for the custom protocol number. The IP header is parsed and then the CLDP header is extracted to determine the message type and payload.

4 Server Implementation

The CLDP server (see file `cldp_server.c`) performs the following:

- Broadcasts HELLO packets every 10 seconds to announce its active status.
- Listens for incoming QUERY messages (`msg_type 0x02`) on the raw socket.
- Processes queries by checking the query type (e.g., for hostname or timestamp) and replies with a RESPONSE (`msg_type 0x03`) packet containing the requested metadata.

The server code includes functions for IP header checksum calculation, debug logging, and manual packet crafting.

5 Client Implementation

The CLDP client (see file `cldp_client.c`):

- Constructs and sends a broadcast QUERY packet (`msg_type 0x02`) requesting metadata (hostname and system time).
- Listens on the raw socket for RESPONSE messages from active servers.
- Collects responses over a configurable time window (e.g., 3 seconds) and displays the received metadata.

6 Demonstration Scenario

The demonstration involves:

1. Running multiple instances of the server. Each server broadcasts a HELLO message every 10 seconds.
2. Running the client to send a broadcast QUERY that requests both hostname and timestamp.
3. Each active server responds with a RESPONSE packet containing the requested meta-data.
4. Traffic capture using tools like `tcpdump` or `Wireshark` with a filter for IP protocol number 253.

7 Build and Run Instructions

7.1 Compiling the Code

Use `make` to build the client and server:

```
gcc -o client cldp_client.c
gcc -o server cldp_server.c
```

Running raw packets require `sudo` access (elevated privileges)

7.2 Running the Server and Client

- To start the server:

```
sudo ./server
```

- To start the client:

```
sudo ./client
```

7.3 Capturing Traffic

To capture CLDP traffic using `tcpdump`, execute:

```
sudo tcpdump -i lo "ip proto 253" -X
```

Replace `lo` with the appropriate interface if needed.

8 Assumptions and Limitations

- The protocol is designed for a controlled network environment.
- Running the client and server on the same machine causes duplicate packet receptions due to loopback routing.
- "Hello" messages (server ip-*i* broadcast address) is captured on enp3s0 interface(filter: ip.proto==253)
- Client to server queries and replies from queries are captured on loopback interface due to the fact that server and client are running on same ip address, the operating system short-circuits the network stack and sends packets over the loopback interface (lo) instead of actually transmitting them on the physical network interface.
- Raw socket programming requires elevated privileges (e.g., running with `sudo`).

9 Conclusion

The CLDP implementation meets the assignment requirements by manually crafting and parsing IP packets using raw sockets. The protocol supports HELLO, QUERY, and RESPONSE message types for node discovery and metadata exchange. This document, along with the source code (`cldp_server.c` and `cldp_client.c`) .